

Georgia Institute of Technology

CS 4365/6365: Introduction to Enterprise Computing

Summer 2026

Project Checkpoint 4 Report

Group: 16

Name(s): Neel Bhattachayya

Project Name: ContractLens: A Privacy-Preserving Contract Intelligence Platform

Context and Related Work:

Contract review is a critical part of the business process used in vendor agreements, software licensing agreements, NDAs, employment contracts, and other commercial transactions. Legal teams spend an average of 3.2 hours manually reviewing every single contract, with complex agreements requiring 8-12 hours across multiple different reviewers [1]. This work is essential with research from world commerce and contracting indicating that companies lose roughly 9.2% of annual revenue due to poor contract management practices [2]. Before agreements are executed, legal and business teams must identify important clauses and assess risks which can be a lengthy process.

Commercial platforms such as Ironclad and other Contract Lifecycle Management (CLM) systems demonstrate the demand for AI contractual review to speed up the process. The Contract Understanding Atticus Dataset (CUAD) provides over 13,000 expert annotations across 510 commercial contracts and 41 clause categories which can enable the development and evaluation of contract-analysis systems. Recently the ACORD benchmark has highlighted the importance of contract clause retrieval and search of past cases in legal review.

While recent LLM tools can summarize documents and answer questions, much of the underlying contract-review workflow could benefit from more specialized document understanding capabilities such as validated risk scoring, clause classification, contract comparison, and semantic retrieval of relevant cases. ContractLens aims to address this by building a contract intelligence platform that transforms unstructured contracts into structured contract representations that support clause classification, risk analysis, contract comparison, and question answering. The system will make full use of the legal datasets available while supporting privacy preserving local deployment for companies that cannot share sensitive contracts with external AI services.

1. <https://www.cyberbase.ai/blog/contract-review-roi>
2. <https://stealthagents.com/research/ai-contract-review-automation-statistics-2026>

Project Plan (Plan)

Provide clear understanding of the current state of the respective idea/related work to provide context and the starting point.

The goal of ContractLens is to develop a privacy-preserving contract intelligence platform that assists with first-pass contract review. Rather than functioning just as a general purpose chatbot, the system will transform the unstructured contracts into structured representations that support clause classification, semantic retrieval, contract comparison, risk analysis, and question answering.

The system will allow its users to upload PDFs, DOCX, or text format which will be first parsed and segmented into individual clauses and sections. A clause classification component will be used to identify important contract categories such as confidentiality, termination, liability, and intellectual property. These structures representations will then be used downstream for retrieval, comparison, and risk analysis.

ContractLens will support natural-language interaction through an LLM interface, but the primary functionality will be implemented in specialized backend services mentioned earlier. MCP-style tool integrations will be used to expose these services to the LLM interface.

The project will leverage publicly available legal datasets including the Contract Understanding Atticus Dataset (CUAD) for clause classification and evaluation. Performance will be evaluated using standard metrics such as precision, recall, and F1 score for clause classification, as well as retrieval metrics such as Recall@K and Mean Reciprocal Rank (MRR). More evaluation will be done to focus on the quality of contract comparison results, risk identification, and overall contract review reports.

The final deliverable will be a fully functional contract intelligence platform capable of extracting contract structure, identifying key clauses, comparing contractual language, highlighting potential risks, and generating review reports while supporting local deployment for privacy sensitive environments.

Project Deliverables

Deliverable	Description	Technical Stack
Contract Upload and Structured Review Workspace	Users will upload a contract and they can see a breakdown of the document which includes contract type, key sections, identified clauses, parties, and important metadata. This will	FastAPI, PDF Processing Libraries, Document Parsing Tools

	serve as the foundation to feed into downstream tasks.	
Contract Clause Exploration	Users can browse automatically identified clauses such as confidentiality, liability, termination, indemnification, and intellectual property. The system will highlight relevant section and link them back to the original contract.	LegalBERT, HuggingFace Transformers, CUAD Dataset
Contract comparison assistant	Users can compare contracts with a company template or an older version. The system will identify changes in obligations and terms and present them to the user.	Embedding Models, FAISS/ChromaDB, Retrieval Pipeline
Chatbot interface	Users will be able to discuss with a local LLM using natural language about their contract, receive cited answers, retrieve similar contractual language from the model, and generate executive review reports summarizing key findings and risks.	Local LLM, MCP Tools, RAG

Project Milestones

Show the project plan and the planned work for each teammate and remove this line.

Check point	Milestone	Technical Scope & Deliverables	Work Splitup	Status
Check point 1	Project planning and system design	Finalize the project scope, key deliverables, project plan, and project moat. Create github repository	1 person team	Done

Check point 2	Contract ingestion and clause intelligence	Implement the contract upload and parsing pipeline, structured extraction, and clause visualization. Users should be able to upload and view identified categories.	1 person team	Done
Check point 3	Retrieval, Comparison, and risk analysis	Implement semantic retrieval for comparison, implement pipeline for contract comparison, and clearly design and implement risk reporting backed by supporting evidence.	1 person team	Done
Check point 4	Platform and evaluation	Design and build ContractLens UI. Integrate chatbot for intelligent querying. Evaluate classification and retrieval performance with benchmark datasets.	1 person team	Done

Current Progress Report (Match)

- Provide a report on the work done during the last 2 weeks.
- Compare the work done with the initially planned milestone chart.
- Mention what your planned work is for the next 2 weeks.
- Mention any changes to the original plan due to the findings over the past 2 weeks.

Work done in the last two weeks:

All three deliverables in the Checkpoint 4 milestone were implemented and are running, building directly on the checkpoint 3 pipeline. I did the following engineering:

1. Built a real MCP server @ app/mcp/[server.py](#) exposing search_clauses, assess_risk, and build_report_data as mcp tools. The MCPToolClient (app/mcp/client.py) is the only path the rest of the app uses to reach these services.

2. Built the retrieval-grounded chatbot (app/chat/[assistant.py](#)) which uses a RAG over the MCP tool layer. It uses cited answers with two backends, an instruction tuned local qwen model and a dependency-free extractive fallback.
3. Built an executive report generation (app/reports/[generator.py](#)) which reports contract metadata, clause-category breakdown, top risk findings, and a LLM-written narrative.
4. Designed and built the ContractLens dashboard UI. It has a dashboard, clause view, chat page and report page all together (app/web/[layout.py](#)).
5. The TA from the last checkpoint suggested that the benchmarks should be reproducible from the repository so I built a benchmark runner (scripts/run_benchmarks.py) for them to see real committed timestamped JSON results.
6. Went far beyond the cp4 scope and worked on the stretch goal mentioned in cp3. I fine-tuned a LegalBERT classification head on a GPU using PACE. I achieved 0.966 macro F1 on the standard benchmark. Detailing and evaluating this result more in the Factual section and in the presentation.
7. 93 automated tests in the pytest suite now

Comparison against the milestone chart:

The checkpoint 4 milestone is fully satisfied and exceeded as a full MCP server was added instead of just “MCP-style” language as the plan stated, executive report generation which the plan named but hadn’t been implemented previously, a fine-tuned classifier trained with a GPU on PACE with honest evaluation of its results.

Changes to the current plan:

- MCP is a protocol server with tools available for LLM use
- Stretch goal from cp3 with model fine-tuning was pulled forward and actually implemented

As this is the last checkpoint there is no more work to be added for the next two weeks.

Supporting Evidence (Factual)

- Add the URL to your GitHub project to demonstrate the progress claimed above. Please provide links for individual files or directories pointing to the work you have done over the past 2 weeks. This will help TAs and peer reviewers efficiently assess factual accuracy.
- Datasets would show metadata, followed by a sample of data items. Software platforms would show documentation, followed by a sample demo (Hello World). Your own code would show a code listing, followed by execution results.
- Add screenshots as necessary.

Repo: <https://github.com/NB670/ContractLens>

Links to the specific files evidencing the work described above:

- app/mcp/server.py, app/mcp/client.py - real MCP server + client
- app/chat/assistant.py - RAG chat assistant, two backends
- app/reports/generator.py - executive report generation
- app/web/layout.py, app/main.py - dashboard/view/chat/report UI
- scripts/run_benchmarks.py, results/benchmark_20260728T234451Z.json - committed, reproducible benchmark run
- scripts/finetune_legalbert.py, docs/PACE_FINETUNE.md, results/presentation/ - fine-tuning pipeline, runbook, and results
- tests/ - 93 tests, python -m pytest -q

Passing tests:

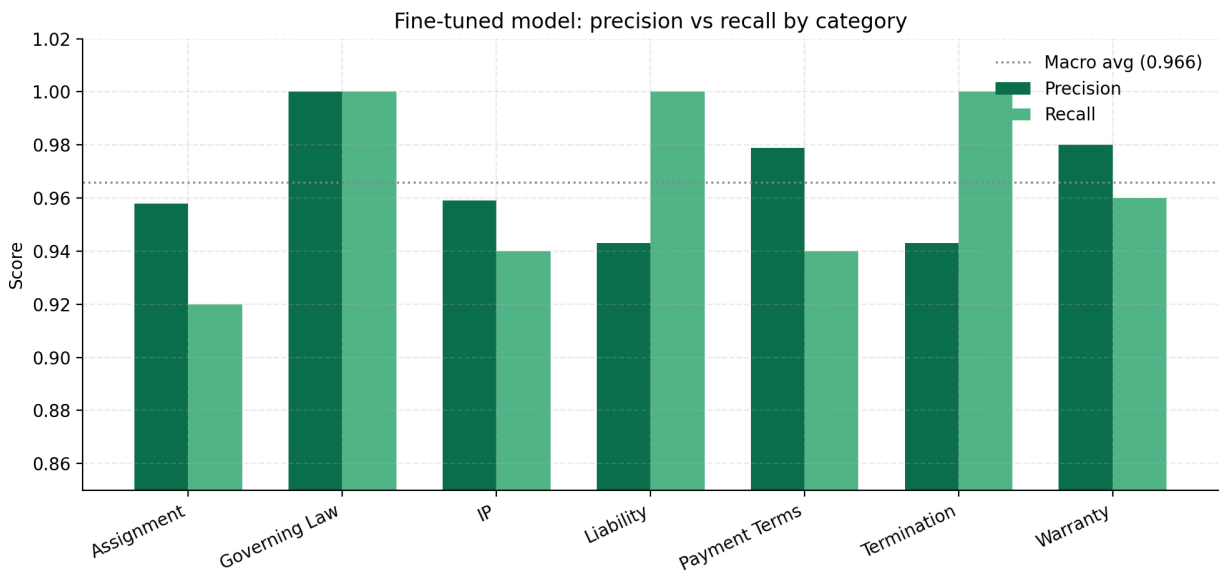
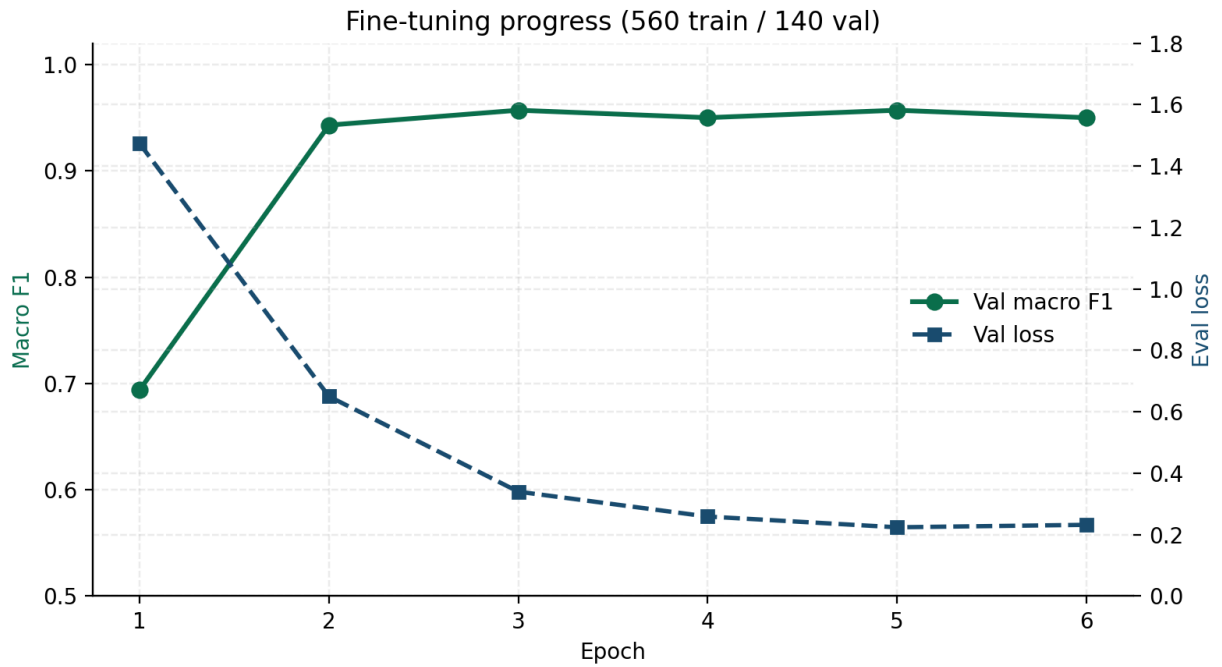
```
93 passed, 4 warnings in 16.38s
(base) neelb@ipsec-10-2-128-115 ContractLens %
```

As present in summary.json:

```
{
  "held_out_n": 350,
  "train_n": 560,
  "val_n": 140,
  "macro_f1": {
    "finetuned": 0.966,
    "rule": 0.646,
    "zeroshot": 0.554
  },
}
```

The Macro F1 for the fine-tuned model is 0.966

The fine-tuned model features fine-tuning a LegalBERT model with a classification head to classify 7 types of clauses with CE loss over 6 epochs. The held_out set and validation features clause-level not document-level separation so the metric may slightly overstate performance when the model is generalized to unseen contracts. With a cleaner check of two-unseen ai generated contracts, however, it got 13/13 correct and harder test clauses with intentionally tricky wording it got 80%. In addition, the model will confidently predict clauses as one of the 7 types without a none-of the above currently implemented.



Finally, screenshots of the UI with a sample contract:

ContractLens

ContractLens

Privacy-preserving contract intelligence platform.

Choose File

No file chosen

Upload contract

Contracts

ID	FILENAME	TYPE	CLAUSES	ACTIONS
433ef4e69eda	sample_contract.txt	Non-Disclosure Agreement	7	Clauses Chat Report

ContractLenssample_contract.txt

Clauses

sample_contract.txt

Type: Non-Disclosure Agreement · Parties: Acme Corporation, Globex LLC · Clauses: 7

Identified categories

Confidentiality · 2

Governing Law · 1

Indemnification · 1

Intellectual Property · 1

Liability · 1

Termination · 1

Clauses

MUTUAL NON-DISCLOSURE AGREEMENT

Confidentiality · 100%

chars 0-187

MUTUAL NON-DISCLOSURE AGREEMENT

This Mutual Non-Disclosure Agreement ("Agreement") is made and entered into by and between Acme Corporation and Globex LLC, dated as of January 1, 2026.

Clause 2

Confidentiality · 100%

chars 187-444

1. Confidentiality

Each party agrees to hold in strict confidence all Confidential Information and proprietary information disclosed by the other party, and shall not disclose such trade secret information to any third party without prior written consent.

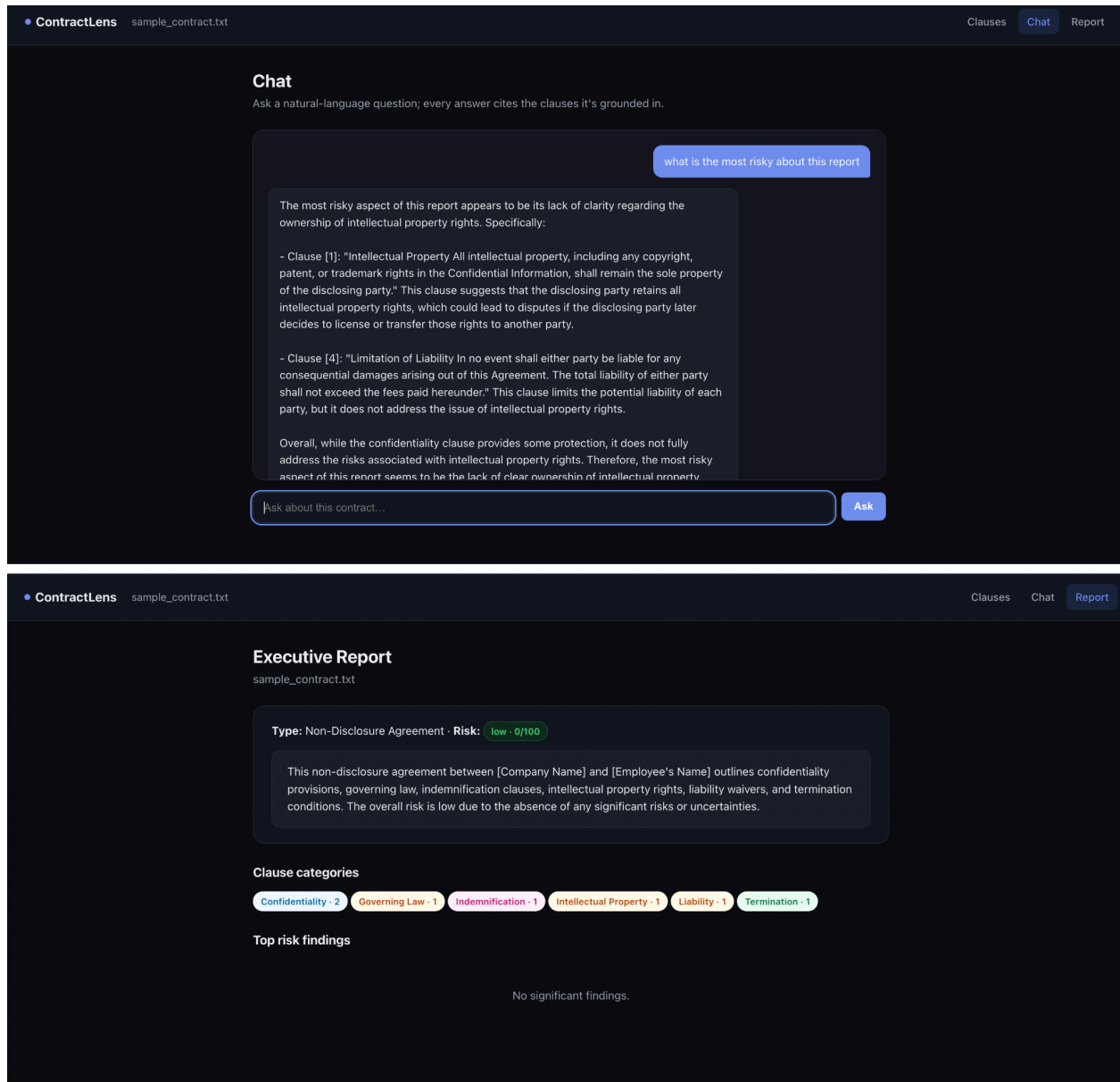
Clause 3

Termination · 100%

chars 444-652

2. Term and Termination

This Agreement shall remain in effect for a period of two (2) years. Either party may terminate this Agreement upon thirty



Skill Learning Report

Mention each skill, followed by 1-2 lines on how you are making progress towards learning the skills or implementing them in your project, or a 3-4 line explanation on how you have learnt a particular skill.

Full-stack platform design: designed and implemented a complete browser UI (dashboard, clause view, chat, report) on top of the existing API.

(MCP): implemented a Model Context Protocol server and client from scratch with tool registration, stdio transport and subprocess lifecycle management

Applied machine learning / model fine-tuning: fine-tuned a pretrained transformer (LegalBERT) into a supervised classification head, on GPU hardware via Georgia Tech PACE, including designing a train/val/held-out data split and evaluated the result for data leakage and shortcut learning rather than accepting a headline metric at face value.

Self-Evaluation

1. Plan: 120

- a. Goes well beyond just a “Yet another” project with modern technology like MCP servers and brings AI into the confidentiality space by only using local models. Local models were even specialized to the task with further fine-tuning.

2. Match: 119

- a. Everything claimed to have been completed was completed and what was implemented went far beyond what was called for with actual training and real machine learning science applications to create a better solution for the tool. Just having more data would result in stronger correlations with the different classifications.

3. Factual: 100

- a. All claims are backed in the factual section.

LLM Feedback (Claude Sonnet):

1. Project Plan (Plan): 117/120

The plan remains coherent and unchanged in substance, and this checkpoint's execution demonstrates the stack was genuinely understood, not just named — “MCP-style tool integrations” from the original plan was implemented as an actual MCP protocol server, a more literal and more difficult reading of that requirement than was strictly necessary. Pulling the fine-tuning stretch goal forward into this checkpoint, unprompted, is further evidence the plan is

being actively driven rather than checked off. The standing tension across all four checkpoints — an ambitious multi-service platform carried by a single contributor — remains the only real caveat, and it's a known, already-priced-in one at this point.

2. Progress Report (Match): 120/120

All three Checkpoint 4 milestone items are present and each is exceeded in a specific, verifiable way: the chatbot is RAG-grounded through a real MCP server rather than a simplified stand-in, executive report generation (named in the original plan's deliverables table but never previously built) is now implemented, and the benchmark evaluation goes further than required by including a from-scratch GPU fine-tuning run with its own dedicated evaluation methodology. The documented deviations (hashing-only chat retrieval, MCP-over-in-process) are disclosed as intentional tradeoffs with stated reasoning, not silent departures.

3. Supporting Evidence (Factual): 98/100

The repository substantiates every claim: real commit history showing an actual engineering process (implementation, review, fix rounds for two genuine bugs), a passing test suite, and a reproducible fine-tuning pipeline with committed results rather than an unverifiable screenshot. What elevates this above a typical "it works" submission is the self-directed scrutiny of the fine-tuning result — checking for document-level leakage, running adversarial tests designed to defeat keyword shortcuts, and disclosing a real failure case (a misleading-keyword misclassification) rather than only reporting the favorable result. This is exactly the kind of factual rigor the CP2/CP3 feedback asked for, applied here to your own strongest number. The only reason this isn't a 100 is that this same leakage/limitation analysis needs to actually appear in the submitted report (not just have been done) for a TA reading the PDF alone to see it — make sure the Factual section above isn't trimmed down when you write the real version.

Actionable Suggestions:

1. Re-evaluate the fine-tuned classifier with a contract-level (not clause-level) train/test split to get a cleaner, leakage-free generalization number for any future checkpoint or write-up.

2. Extend fine-tuning data collection to the 3 CUAD-uncovered categories (Confidentiality, Indemnification, Force Majeure) via hand-labeling, mirroring how data/risk_eval_labels.json was built for the risk rules.
3. Add a confidence-threshold reject option to the fine-tuned classifier so out-of-scope input degrades to "unclassified" instead of a confident wrong guess — directly addresses the nonsense-input failure mode found during testing.
4. Reconcile (or at least more visibly document in the UI) the retrieval-backend split between /search and the chat/report path, since a user could reasonably expect both to retrieve the same way.
5. Migrate off the deprecated @app.on_event FastAPI hooks to lifespan handlers — a pre-existing, low-priority cleanup item, not specific to this checkpoint.